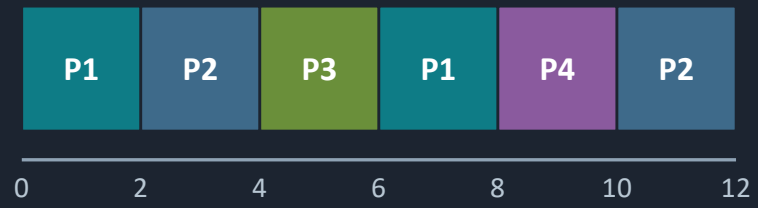
KARADENİZ TEKNİK ÜNİVERSİTESİ

Yazılım Geliştirme Bölümü

İŞLETİM SİSTEMLERİ

Hafta 5

CPU Zamanlama



Hazır kuyruğu → CPU Zamanlayıcısı → Dağıtıcı → CPU

Dr. Öğr. Üyesi Sayyad Alizadeh

Bu Haftanın Yol Haritası

1

Temel kavramlar

Patlama döngüsü, zamanlayıcı, dağıtıcı, kesintili zamanlama

2

Zamanlama ölçütleri

TAT, bekleme ve yanıt süresi nasıl hesaplanır?

3

FCFS ve SJF

Gelme sırası mı, en kısa iş mi?

4

SRTF ve öncelik

Kesme ne zaman, kim için?

5

Round Robin

Zaman dilimi seçimi neyi değiştirir?

6

Karşılaştırma

Aynı veri, yedi çözüm

7

Çok düzeyli kuyruklar

Çok düzeyli kuyruk ve geri beslemeli kuyruk

8

Çok işlemci ve gerçek sistemler

Yakınlık, yük dengeleme · Linux, Windows

Öğrenme Çıktıları

Bu haftanın sonunda öğrenciler:

1. CPU Zamanlayıcısı, dağıtıcı ve Donanım Zamanlayıcısını ayırt eder
2. Kesintili ve kesintisiz zamanlamayı karar noktalarıyla açıklar
3. Tamamlanma, bekleme ve yanıt sürelerini formülleriyle doğru hesaplar
4. FCFS, SJF, SRTF, öncelik ve RR için Gantt şeması çizer ve ortalamaları bulur
5. Algoritmaları aynı iş yükü üzerinde karşılaştırır ve senaryoya göre seçer
6. Çok düzeyli kuyrukları, çok işlemcili zamanlamayı ve gerçek sistem örneklerini açıklar

Önceki Haftalardan Bu Haftaya

DAHA ÖNCE GÖRDÜKLERİMİZ

- Hazır kuyruğu (Hafta 3)
- Bağlam değiştirme (Hafta 3)
- Süreç durumları (Hafta 3)
- Çok iş parçacığı (Hafta 4)



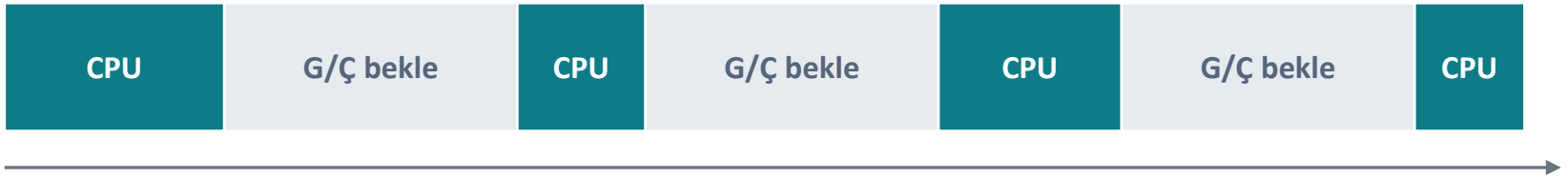
BU HAFTANIN TEMEL SORUSU

Hazır kuyruğunda birden fazla süreç ya da iş parçacığı varsa CPU'yu sıradaki kime vereceğiz?

Bu hafta: bu kararı veren algoritmalar ve onları karşılaştırma ölçütleri

Not: Kitapta olduğu gibi "süreç" ifadesi, çekirdek düzeyi iş parçacıklarını da kapsayacak biçimde kullanılır.

CPU–G/Ç Patlama Döngüsü (CPU–I/O Burst Cycle)



CPU patlaması → G/Ç patlaması → CPU patlaması → … Süreç yürütmesi bu iki durum arasında gidip gelir.

G/Ç ağırlıklı süreç

Çok sayıda kısa CPU patlaması; zamanının çoğu G/Ç beklemekle geçer (ör. etkileşimli programlar).

CPU ağırlıklı süreç

Az sayıda uzun CPU patlaması (ör. büyük hesaplamalar).

Amaç: Bir süreç G/Ç beklerken CPU boş kalmamalı; hazır süreçlerden uygun biri seçilmeli. Ölçümlerde kısa CPU patlamaları çok, uzun patlamalar azdır.

Kaynak: Silberschatz vd., Bölüm 6.1.1 (Şekil 6.1–6.2)

CPU Zamanlayıcısı (CPU Scheduler)

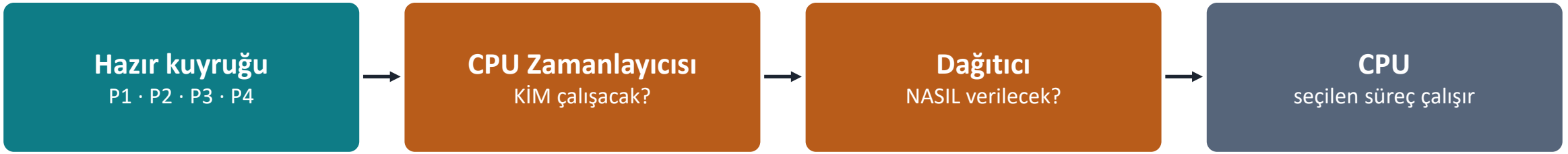
CPU Zamanlayıcısı: CPU boşaldığında hazır kuyruğundaki süreçlerden birini seçen İS bileşenidir (kısa vadeli zamanlayıcı).

- Hazır kuyruğu FIFO olmak zorunda değildir: öncelik kuyruğu, ağaç ya da sırasız bağlı liste olabilir
- Kuyruktaki kayıtlar genellikle süreçlerin PCB'leridir
- Seçimin nasıl yapılacağını zamanlama algoritması belirler — bu haftanın konusu

Ölçüt	CPU Zamanlayıcısı (CPU Scheduler)	Donanım Zamanlayıcısı (Timer)
Türü	İS'nin yazılım bileşeni	Donanım bileşeni
Görevi	Hangi sürecin çalışacağına karar verir	Belirlenen süre dolunca kesme üretir
İlişki	Donanım Zamanlayıcısı kesmesi İS'ye denetimi verir → İS gerekirse CPU Zamanlayıcısını çalıştırır	

Kaynak: Silberschatz vd., Bölüm 6.1.2 · Hafta 1 ayrımının hatırlatması

Dağıtıcı (Dispatcher): Seçilen Sürece CPU'yu Vermek



Dağıtıcının görevleri

1. Bağlam değiştirme (eski sürecin durumunu kaydet, yenisininkini yükle)
2. Kullanıcı moduna geçiş
3. Programı kaldığı doğru noktadan yeniden başlatmak için uygun konuma atlama

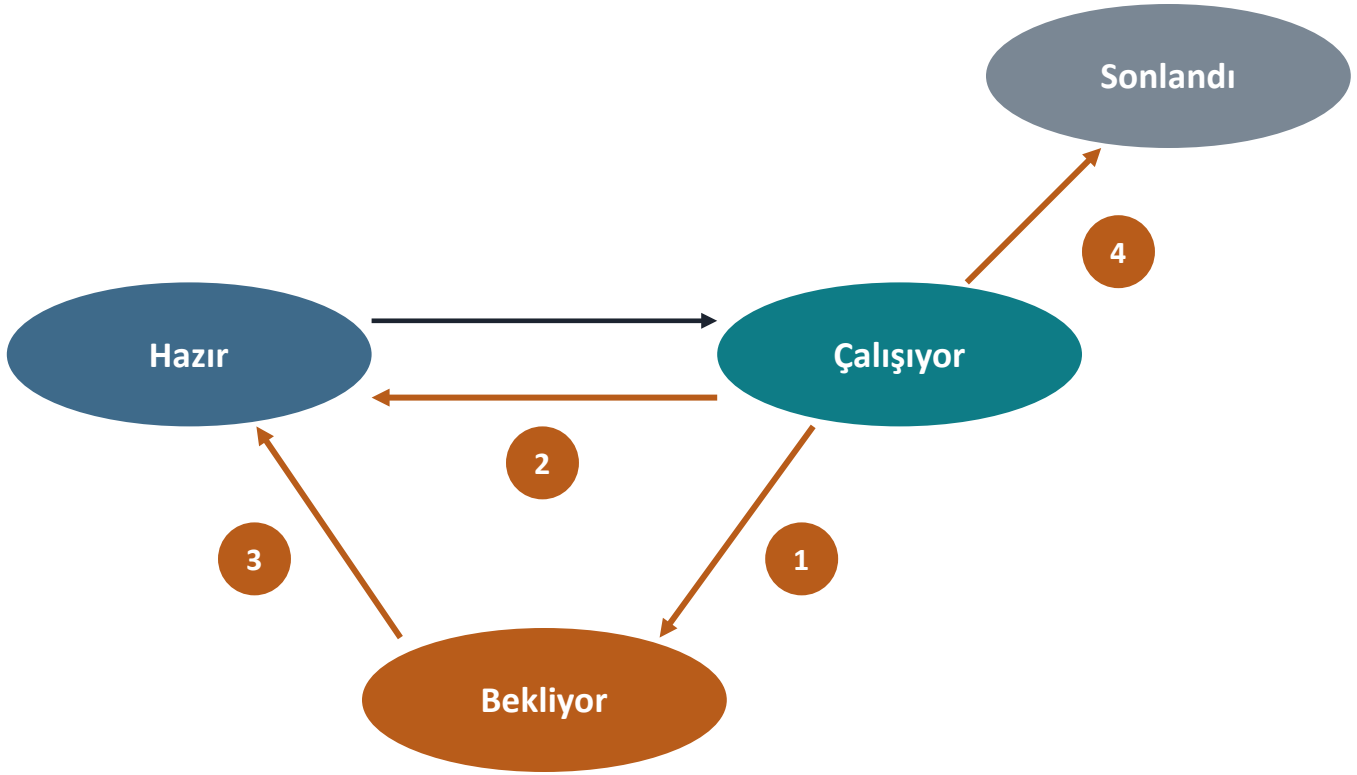
Dağıtım gecikmesi (dispatch latency)



Bir süreci durdurup diğerini başlatmanın aldığı süre. Her süreç geçişinde çalıştığı için dağıtıcı olabildiğince hızlı olmalıdır.

Kaynak: Silberschatz vd., Bölüm 6.1.4

Zamanlama Kararı Ne Zaman Gerekir?



1	**Çalışıyor → Bekliyor** G/Ç isteği, wait() çağrısı	kesintisiz
2	**Çalışıyor → Hazır** kesme (ör. süre doldu)	kesintili
3	**Bekliyor → Hazır** G/Ç tamamlandı	kesintili
4	**Süreç sonlanır** exit()	kesintisiz

Yalnızca 1 ve 4'te zamanlama yapılıyorsa → kesintisiz (nonpreemptive). 2 ve 3'te de yapılabiliyorsa → kesintili (preemptive).

Kesintili ve Kesintisiz Zamanlama

Kesintisiz Zamanlama

Nonpreemptive Scheduling

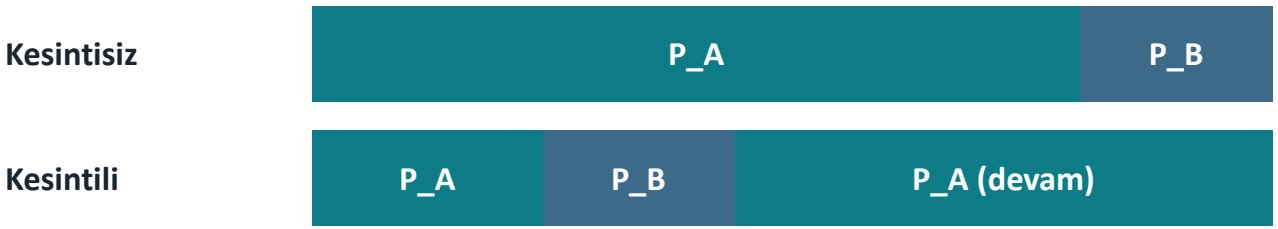
- Süreç CPU'yu sonlanana ya da beklemeye geçene kadar tutar
- Ek donanım (Donanım Zamanlayıcısı) gerektirmez
- Uzun bir süreç diğerlerini uzun süre bekletebilir

Kesintili Zamanlama

Preemptive Scheduling

- İS çalışan süreçten CPU'yu geri alabilir
- Donanım Zamanlayıcısı kesmesi buna olanak verir
- Yanıt süresi iyileşir; ama paylaşılan veride tutarlılık sorunları doğabilir (Hafta 6)

Aynı iki süreç: P_A (uzun) çalışırken P_B (kısa) gelir



Kesintili zamanlamada P_B daha erken CPU alır; bedeli fazladan bir bağlam değiştirmedir.

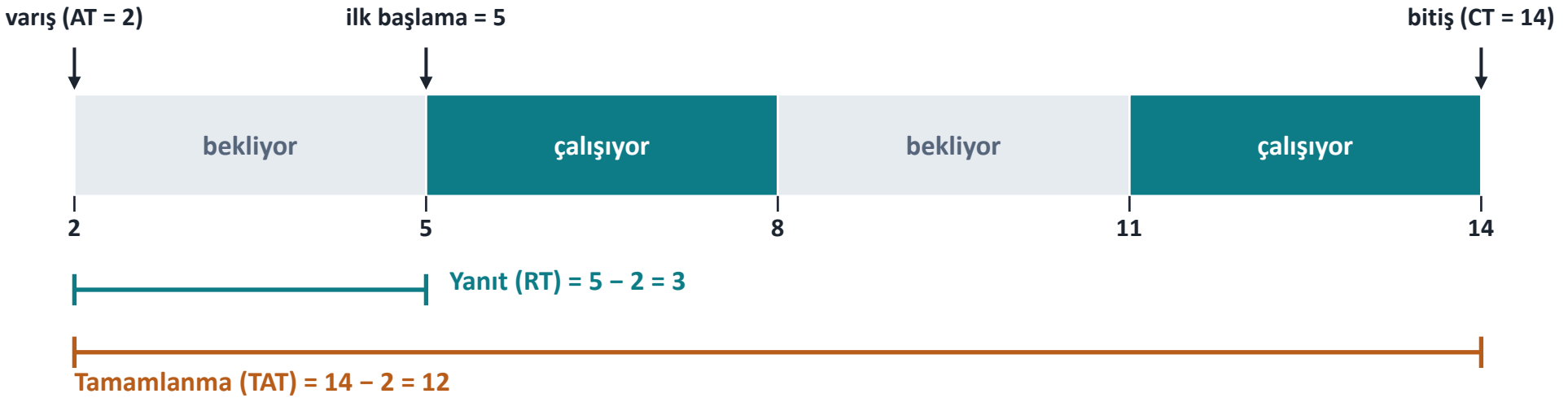
Kaynak: Silberschatz vd., Bölüm 6.1.3 · Şema kavramsaldır

Zamanlama Ölçütleri (Scheduling Criteria)

CPU kullanımı CPU Utilization	CPU'nun meşgul olduğu zaman oranı	**en büyükle**
Verim Throughput	Birim zamanda tamamlanan süreç sayısı	**en büyükle**
Tamamlanma süresi Turnaround Time	Varıştan bitişe kadar geçen toplam süre	**en küçükle**
Bekleme süresi Waiting Time	Hazır kuyruğunda bekleyerek geçen toplam süre	**en küçükle**
Yanıt süresi Response Time	Varıştan ilk kez CPU almaya kadar geçen süre	**en küçükle**

Bekleme süresi yalnızca hazır kuyruğunda geçen zamanı sayar; algoritma, G/Ç'de geçen süreyi ya da CPU'da çalışma süresini değiştirmez, yalnızca bekleme süresini etkiler.

Üç Süreyi Karıştırmayın: Bir Süreç Üzerinde Gösterim



Bekleme (WT) = 3 + 3 = 6 · kontrol: TAT − BT = 12 − 6 = 6 (BT = 6: iki çalışma parçası)

TAT = CT − AT
Tamamlanma süresi

WT = TAT − BT
Bekleme süresi (BT: toplam CPU patlaması)

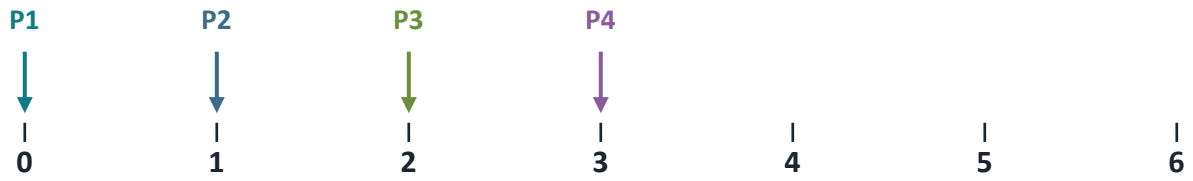
RT = İlk başlama − AT
Yanıt süresi

Bu hafta süreçlerin G/Ç yapmadığı varsayılır: WT, hazır kuyruğunda geçen süreye eşittir. Örnekteki ara bekleme, kesintili zamanlamayı temsil eder.

Bu Haftanın Ortak İş Yükü

Süreç	Varış zamanı (AT)	CPU patlaması (BT)	Öncelik
P1	0	8	2
P2	1	4	1
P3	2	9	3
P4	3	5	2

Varışlar



Kurallar (tüm çözümlerde sabit)

Öncelik — Küçük sayı = yüksek öncelik (P2 en yüksek)

Eşitlik — Eşit önceliklerde ya da eşit sürelerde önce gelen (FCFS)

Round Robin — Bir zaman dilimi biterken aynı anda gelen yeni süreç, kesilen süreçten önce kuyruğa eklenir

Varsayım — G/Ç yok; bağlam değiştirme süresi 0 kabul edilir

Aynı veriyi beş algoritmanın yedi çeşidiyle çözüp karşılaştıracağız: bu bir deterministik modellemedir.

FCFS: İlk Gelen İlk Hizmet Alır (First-Come, First-Served)

- CPU'yu ilk isteyen süreç ilk alır
- Hazır kuyruğu FIFO kuyruğudur: gelen süreç kuyruğun sonuna eklenir
- Kesintisizdir: süreç CPU'yu bitene ya da G/Ç isteyene kadar tutar
- Gerçekleştirmesi en basit algoritmadır

FIFO hazır kuyruğu



son (yeni gelen buraya eklenir) … baş (CPU'ya ilk gider)

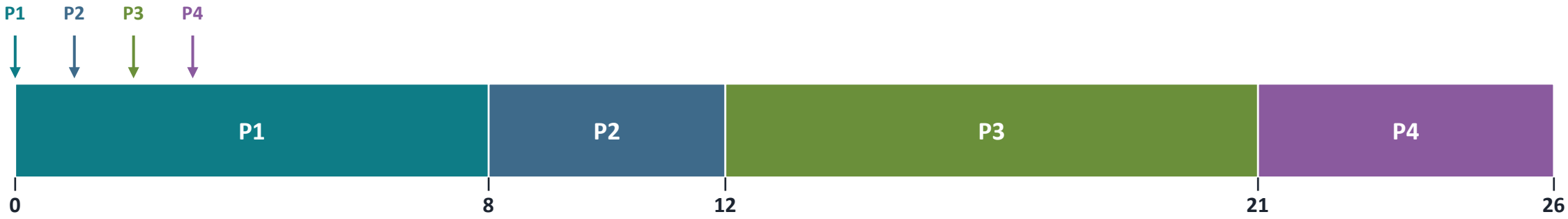
Zayıf yanı

Ortalama bekleme süresi çoğu zaman uzundur ve sürelerin gelme sırasına göre çok değişir.

Etkileşimli ve zaman paylaşımlı sistemler için uygun değildir: her kullanıcının düzenli aralıklarla CPU alması gerekir.

Kaynak: Silberschatz vd., Bölüm 6.3.1; Tanenbaum & Bos, Bölüm 2.4.2

FCFS: Ortak Veri Üzerinde Gantt Şeması



ADIM ADIM

t = 0 Yalnızca P1 vardır → P1 başlar, 8 birim çalışır (0–8)

t = 8 Kuyruk gelme sırasıyla: P2, P3, P4 → P2 çalışır (8–12)

t = 12 P3 çalışır (12–21)

t = 21 P4 çalışır (21–26)

Dikkat

P2 t=1'de gelir ama P1 kesilmez: FCFS kesintisizdir. P2 ancak P1 bitince (t=8) CPU alır.

Üstteki oklar varış zamanlarını gösterir. Hesap tablosu bir sonraki slaytta.

FCFS: Süre Hesapları



Süreç	Varış (AT)	Patlama (BT)	İlk başlama	Bitiş (CT)	TAT = CT − AT	WT = TAT − BT	RT = İlk − AT
P1	0	8	0	8	8	0	0
P2	1	4	8	12	11	7	7
P3	2	9	12	21	19	10	10
P4	3	5	21	26	23	18	18
Ortalama					**15,25**	**8,75**	**8,75**

Ort. TAT = (8 + 11 + 19 + 23) / 4 = 15,25

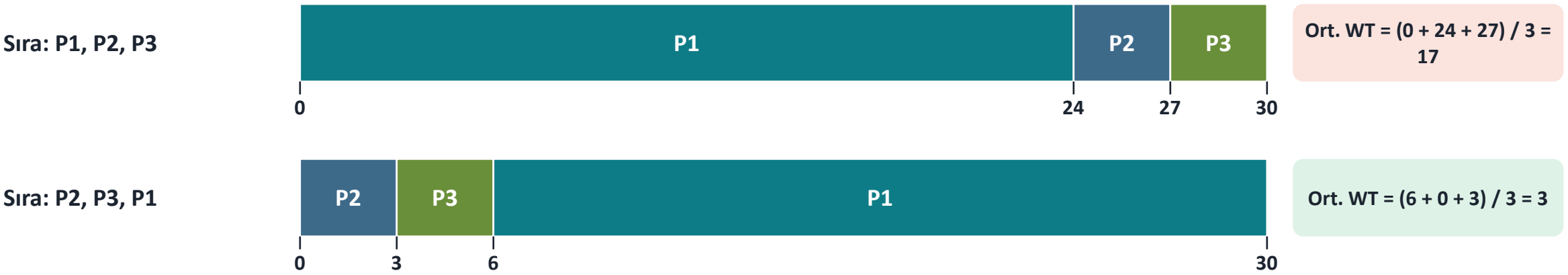
Ort. WT = (0 + 7 + 10 + 18) / 4 = 8,75

Ort. RT = (0 + 7 + 10 + 18) / 4 = 8,75

Kesintisiz algoritmalarda her süreç bir kez başladığı için RT = WT olur.

Konvoy Etkisi (Convoy Effect)

Üç süreç t = 0'da gelir: P1 = 24, P2 = 3, P3 = 3 (CPU patlaması)



Konvoy etkisi nedir?

Kısa süreçlerin, CPU'yu tutan tek bir uzun sürecin arkasında sıra beklemesi. CPU ağırlıklı süreç önde olunca G/Ç ağırlıklı kısa süreçler ve aygıtları boşta kalır.

Sonuç

FCFS'de ortalama bekleme süresi, gelme sırasına bağlı olarak büyük değişebilir: aynı iş yükü için 17 ile 3.

Kaynak: Silberschatz vd., Bölüm 6.3.1 (kitap örneği)

SJF: En Kısa İş Önce (Shortest-Job-First)

- CPU boşalınca, hazır kuyruğundaki süreçlerden bir sonraki CPU patlaması en kısa olan seçilir
- Eşitlikte FCFS kullanılır
- Bu bölümdeki sürüm kesintisizdir: seçilen süreç patlaması bitene kadar çalışır
- Daha doğru ad: “en kısa sonraki CPU patlaması”

Kritik kural

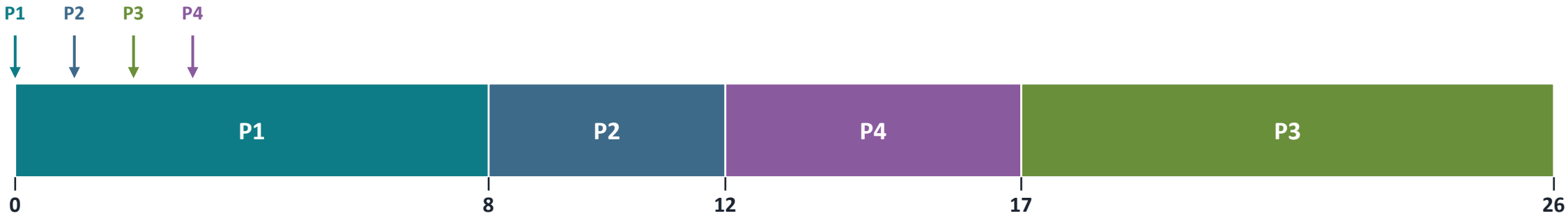
Yalnızca o anda varmış süreçler seçilebilir. Henüz gelmemiş bir süreç ne kadar kısa olursa olsun seçilemez.

Ortak veride: t = 0'da yalnızca P1 vardır → P1 seçilir (P2 daha kısa ama t = 1'de gelir).

SJF, verilen bir süreç kümesi için en küçük ortalama bekleme süresini veren algoritmadır (kısa işi öne almak, uzun işin beklemesini kısa işin kazandığından daha az artırır).

Kaynak: Silberschatz vd., Bölüm 6.3.2; Tanenbaum & Bos, Bölüm 2.4.2

SJF (Kesintisiz): Ortak Veri Üzerinde Gantt Şeması



KARAR ANLARI

t = 0 Gelmiş süreç: yalnızca P1 → P1 (0–8)

t = 8 Gelmiş süreçler: P2 = 4, P3 = 9, P4 = 5 → en kısa P2 (8–12)

t = 12 Kalanlar: P3 = 9, P4 = 5 → P4 (12–17)

t = 17 Yalnızca P3 → P3 (17–26)

FCFS'ten farkı

t = 8'de sıra P3 değil P4'tür: P4 (5) P3'ten (9) kısadır.

SJF (Kesintisiz): Süre Hesapları



Süreç	Varış (AT)	Patlama (BT)	İlk başlama	Bitiş (CT)	TAT = CT − AT	WT = TAT − BT	RT = İlk − AT
P1	0	8	0	8	8	0	0
P2	1	4	8	12	11	7	7
P3	2	9	17	26	24	15	15
P4	3	5	12	17	14	9	9
Ortalama					**14,25**	**7,75**	**7,75**

Ort. TAT = (8 + 11 + 24 + 14) / 4 = 14,25

Ort. WT = (0 + 7 + 15 + 9) / 4 = 7,75

Ort. RT = (0 + 7 + 15 + 9) / 4 = 7,75

FCFS ile karşılaştırma: ort. WT 8,75 → 7,75. Kesintisiz olduğundan RT = WT.

SJF Optimaldir — Ama Bir Sorun Var

Kuramsal güç

Verilen süreç kümesi için ortalama bekleme süresini en aza indirir. Bu yüzden diğer algoritmaları değerlendirmek için bir ölçüt olarak da kullanılır.

Pratik sorun

CPU Zamanlayıcısı, bir sürecin bir sonraki CPU patlamasının ne kadar süreceğini önceden bilemez.

Çözüm yolları

Uzun vadeli (toplu iş) zamanlamada
Kullanıcının iş gönderirken belirttiği süre sınırı kullanılabilir

CPU zamanlamada
Bir sonraki patlama bilinemez → geçmiş patlamalardan tahmin edilir

Varsayım: sonraki patlama, öncekilere benzer uzunlukta olacaktır → üstel ortalama ile tahmin

Sonraki CPU Patlamasının Tahmini: Üstel Ortalama

$$\tau(n+1) = \alpha \cdot t(n) + (1 - \alpha) \cdot \tau(n)$$

$t(n)$	gerçekleşen son (n.) CPU patlaması
$\tau(n)$	önceki tahmin (geçmişin özeti)
α	$0 \le \alpha \le 1$: yakın geçmişin ağırlığı

$\alpha = 0 \rightarrow$ yalnızca geçmiş tahmin · $\alpha = 1 \rightarrow$ yalnızca son patlama · yaygın seçim $\alpha = 1/2$

Çözümlü örnek

Verilenler: $\alpha = 0{,}5$ · $\tau(n) = 10$ ms · $t(n) = 6$ ms

$$\tau(n+1) = 0{,}5 \cdot 6 + (1 - 0{,}5) \cdot 10$$

$$\tau(n+1) = 3 + 5$$

$$\tau(n+1) = 8 \text{ ms}$$

Yorum: son patlama (6) tahminden (10) kısa çıktı; yeni tahmin ikisinin ortasına (8) çekildi.

Kaynak: Silberschatz vd., Bölüm 6.3.2 (Şekil 6.3)

SRTF: En Kısa Kalan Süre Önce (Shortest-Remaining-Time-First)

- SJF'nin kesintili sürümüdür
- Yeni bir süreç geldiğinde, onun patlaması çalışan sürecin KALAN süresiyle karşılaştırılır
- Yeni süreç daha kısaysa çalışan süreç kesilir ve hazır kuyruğuna döner
- Karşılaştırmada ilk patlama değil, kalan süre (remaining time) kullanılır

Karar ne zaman verilir?

- Yeni bir süreç geldiğinde
- Çalışan süreç bittiğinde

Diğer anlarda çalışan süreç devam eder.

SJF ve SRTF aynı fikrin iki sürümüdür: kesintisiz SJF yalnızca CPU boşalınca seçer; SRTF her varışta yeniden değerlendirir.

Kaynak: Silberschatz vd., Bölüm 6.3.2; Tanenbaum & Bos, Bölüm 2.4.2 (shortest remaining time next)

SRTF: Karar Anları Adım Adım

t = 0 Hazır: P1 (kalan 8) → P1 başlar

t = 1 P2 gelir (4). P1'in kalanı 7. 4 < 7 → P1 KESİLİR, P2 başlar

t = 2 P3 gelir (9). P2'nin kalanı 3. 9 > 3 → P2 devam

t = 3 P4 gelir (5). P2'nin kalanı 2. 5 > 2 → P2 devam

t = 5 P2 biter. Hazır: P1 = 7, P3 = 9, P4 = 5 → en kısa P4

t = 10 P4 biter. Hazır: P1 = 7, P3 = 9 → P1

t = 17 P1 biter. Yalnızca P3 (9) → P3 (17–26)

Kalan süreler

t	P1	P2	P3	P4
0	8	–	–	–
1	7	4	–	–
2	7	3	9	–
3	7	2	9	5
5	7	0	9	5
10	7	0	9	0
17	0	0	9	0

Tabloda her karar anındaki kalan süreler: seçim her zaman o anda gelmiş süreçlerin kalan süresine göredir.

SRTF: Gantt Şeması



OKUMA İPUÇLARI

- P1 iki parçada çalışır: 0–1 ve 10–17 (toplam 8 birim)
- Tek kesme t = 1'dedir: P2'nin 4'ü, P1'in kalan 7'sinden kısadır
- t = 2 ve t = 3'teki varışlar kesmeye yol açmaz: P2'nin kalanı daha kısadır
- P4, t = 5'te P1'den önce seçilir: kalan 5 < kalan 7

Bağlam değiştirme

4

5 parça → 4 geçiş (SJF'de 3)

SRTF: Süre Hesapları



Süreç	Varış (AT)	Patlama (BT)	İlk başlama	Bitiş (CT)	TAT = CT − AT	WT = TAT − BT	RT = İlk − AT
P1	0	8	0	17	17	9	0
P2	1	4	1	5	4	0	0
P3	2	9	17	26	24	15	15
P4	3	5	5	10	7	2	2
Ortalama					**13**	**6,5**	**4,25**

Ort. TAT = (17 + 4 + 24 + 7) / 4 = 13

Ort. WT = (9 + 0 + 15 + 2) / 4 = 6,5

Ort. RT = (0 + 0 + 15 + 2) / 4 = 4,25

Kesintili olduğundan RT ≠ WT olabilir (P1: RT = 0, WT = 9). SJF ile karşılaştırma: ort. WT 7,75 → 6,5.

Öncelik Zamanlama (Priority Scheduling)

- Her sürece bir öncelik atanır; CPU en yüksek öncelikli hazır sürece verilir
- Eşit önceliklerde FCFS uygulanır
- SJF, önceliğin (tahmini) sonraki patlama olduğu özel bir öncelik algoritmasıdır
- Öncelikler içsel (bellek gereksinimi, açık dosya sayısı…) ya da dışsal (önem, ödeme…) olabilir

Öncelik yönü

Küçük sayının mı büyük sayının mı yüksek öncelik olduğu sisteme göre değişir.

Bu derste: küçük sayı = yüksek öncelik

Ortak veride: P2 (1) > P1 = P4 (2) > P3 (3)

Kesintisiz öncelik

Daha yüksek öncelikli süreç gelse de çalışan süreç devam eder; yeni süreç kuyruğun başına konur.

Kesintili öncelik

Gelen sürecin önceliği çalışandan yüksekse çalışan süreç hemen kesilir.

Kaynak: Silberschatz vd., Bölüm 6.3.3; Tanenbaum & Bos, Bölüm 2.4.3

Kesintisiz Öncelik: Ortak Veri Üzerinde



t = 0 Yalnızca P1 (öncelik 2) → P1 (0–8)

t = 8 Hazır: P2 (1), P3 (3), P4 (2) → en yüksek öncelik P2 (8–12)

t = 12 P3 (3), P4 (2) → P4 (12–17)

t = 17 P3 (17–26)

CT: P1 = 8, P2 = 12, P3 = 26, P4 = 17 · Bu iş yükünde sıra, rastlantı sonucu SJF ile aynı çıkmıştır.

Ort. TAT = (8 + 11 + 24 + 14) / 4 = 14,25

Ort. WT = (0 + 7 + 15 + 9) / 4 = 7,75

Ort. RT = (0 + 7 + 15 + 9) / 4 = 7,75

Ayrıntılı tablo SJF ile özdeştir. Kesintisiz olduğundan RT = WT.

Kesintili Öncelik: Karar Anları



t = 0	Yalnızca P1 (2) → P1 başlar
t = 1	P2 gelir, öncelik 1 > P1'in 2'si → P1 KESİLİR, P2 başlar
t = 2	P3 gelir (3): P2'den düşük → P2 devam
t = 3	P4 gelir (2): P2'den düşük → P2 devam
t = 5	P2 biter. Hazır: P1 (2), P3 (3), P4 (2) → P1 ile P4 eşit; FCFS: P1 daha önce gelmiştir → P1 (5–12)
t = 12	P4 (2), P3 (3) → P4 (12–17); ardından P3 (17–26)

Eşitlik kuralı (FCFS) çözüm boyunca değiştirilmez: varış zamanı erken olan önce seçilir.

Kesintili Öncelik: Süre Hesapları



Süreç	Varış (AT)	Patlama (BT)	İlk başlama	Bitiş (CT)	TAT = CT − AT	WT = TAT − BT	RT = İlk − AT
P1	0	8	0	12	12	4	0
P2	1	4	1	5	4	0	0
P3	2	9	17	26	24	15	15
P4	3	5	12	17	14	9	9
Ortalama					**13,5**	**7**	**6**

Ort. TAT = (12 + 4 + 24 + 14) / 4 = 13,5

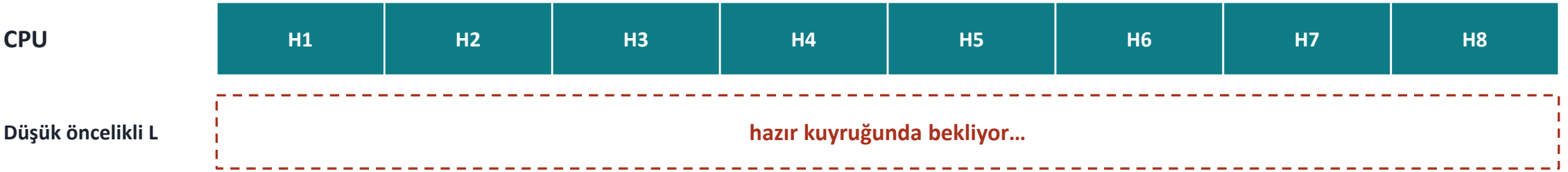
Ort. WT = (4 + 0 + 15 + 9) / 4 = 7

Ort. RT = (0 + 0 + 15 + 9) / 4 = 6

Kesintisiz öncelikle karşılaştırma: ort. WT 7,75 → 7; ort. RT 7,75 → 6.

Açlık (Starvation) ve Yaşlandırma (Aging)

Açlık (belirsiz engellenme): Düşük öncelikli bir süreç, sürekli daha yüksek öncelikli süreçler geldiği için çok uzun süre — hatta hiç — CPU alamayabilir.



H1…H8: art arda gelen yüksek öncelikli süreçler

Çözüm: Yaşlandırma (Aging)

Sistemde uzun süre bekleyen süreçlerin önceliği yavaş yavaş artırılır. Sonunda düşük öncelikli süreç de en yüksek önceliğe ulaşır ve çalışır.

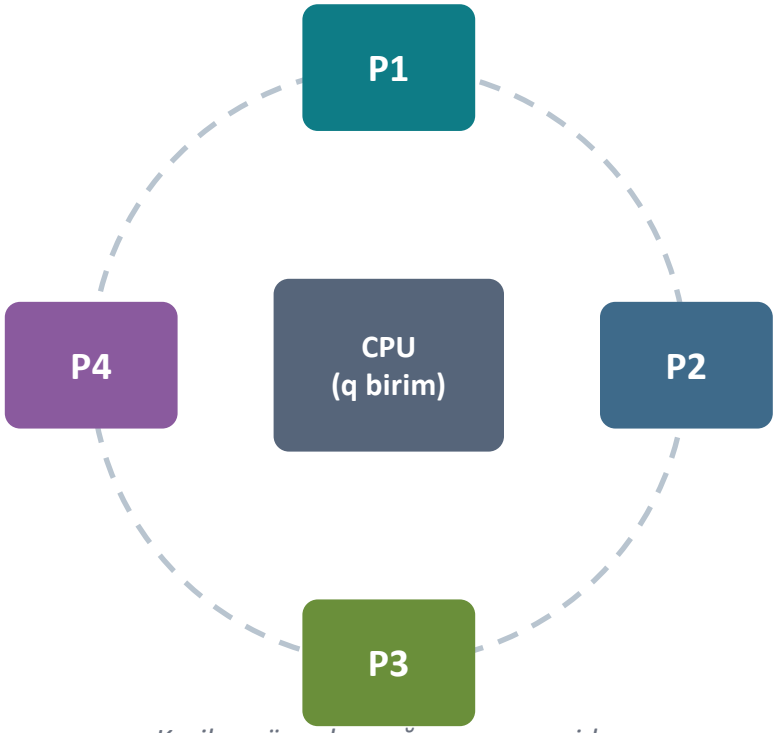
Kitaptaki sayısal örnek

Öncelik 127 (düşük) ile 0 (yüksek) arasında; bekleyen sürecin önceliği her 15 dakikada 1 artırılırsa, 127'den 0'a en fazla 32 saatte ulaşır (127 × 15 dk ≈ 31,75 sa).

Kaynak: Silberschatz vd., Bölüm 6.3.3

Round Robin (RR): Zaman Dilimiyle Sıralı Paylaşım

- Her süreç en fazla bir zaman dilimi (time quantum, q) kadar çalışır
- Hazır kuyruğu dairesel bir FIFO kuyruğudur
- Süreç dilim dolmadan biterse CPU'yu kendisi bırakır
- Dilim dolar ve bitmezse: Donanım Zamanlayıcısı kesmesi → süreç kesilir → kuyruğun SONUNA eklenir
- Kesintilidir; zaman paylaşımlı sistemler için tasarlanmıştır



Kesilen süreç kuyruğun sonuna gider

n süreç ve q dilimiyle hiçbir süreç bir sonraki dilimini (n − 1) × q zaman biriminden fazla beklemez.

RR (q = 2): Hazır Kuyruğu Nasıl Değişir?

Zaman	Çalışan	Kalan	Bu aralıkta gelen	Aralık sonunda hazır kuyruğu (baş → son)
0–2	P1	6	P2, P3	P2 → P3 → P1
2–4	P2	2	P4	P3 → P1 → P4 → P2
4–6	P3	7	–	P1 → P4 → P2 → P3
6–8	P1	4	–	P4 → P2 → P3 → P1
8–10	P4	3	–	P2 → P3 → P1 → P4
10–12	P2	0	–	P3 → P1 → P4
12–14	P3	5	–	P1 → P4 → P3
14–16	P1	2	–	P4 → P3 → P1
16–18	P4	1	–	P3 → P1 → P4
18–20	P3	3	–	P1 → P4 → P3
20–22	P1	0	–	P4 → P3
22–23	P4	0	–	P3
23–25	P3	1	–	P3
25–26	P3	0	–	boş

Kural: t = 2'de P3 gelir ve P1'in dilimi biter → önce P3, sonra P1 kuyruğa eklenir. t = 0'da kuyrukta yalnızca P1 vardır.

RR (q = 2): Gantt Şeması



OKUMA İPUÇLARI

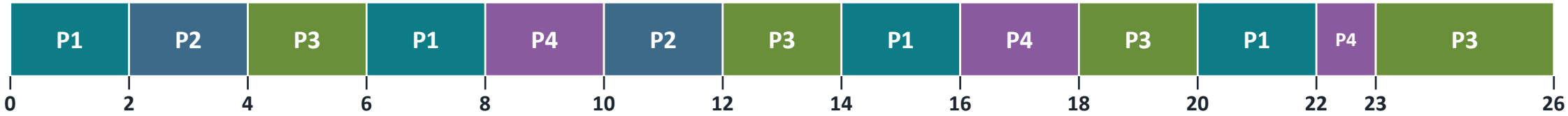
- Her parça en fazla 2 birimdir; P2 (10–12), P1 (20–22) ve P4 (22–23) son parçalarında biter
- P4'ün son parçası 1 birimdir: kalanı 1'dir, dilimi doldurmadan CPU'yu bırakır
- P3, 23–26 arasında tek başına kaldığı için dilim dolsa da yeniden seçilir (23–25, 25–26)
- Aynı süreç farklı aralıklarda aynı renkle gösterilir

Bağlam değiştirme

12

Farklı süreçler arasındaki geçiş sayısı (13 parça → 12 geçiş)

RR (q = 2): Süre Hesapları



Süreç	Varış (AT)	Patlama (BT)	İlk başlama	Bitiş (CT)	TAT = CT − AT	WT = TAT − BT	RT = İlk − AT
P1	0	8	0	22	22	14	0
P2	1	4	2	12	11	7	1
P3	2	9	4	26	24	15	2
P4	3	5	8	23	20	15	5
Ortalama					**19,25**	**12,75**	**2**

Ort. TAT = (22 + 11 + 24 + 20) / 4 = 19,25

Ort. WT = (14 + 7 + 15 + 15) / 4 = 12,75

Ort. RT = (0 + 1 + 2 + 5) / 4 = 2

RT çok küçüktür (her süreç kısa sürede ilk kez CPU alır); TAT ve WT ise FCFS/SJF'den büyüktür.

RR (q = 4): Kuyruk İzi ve Gantt Şeması



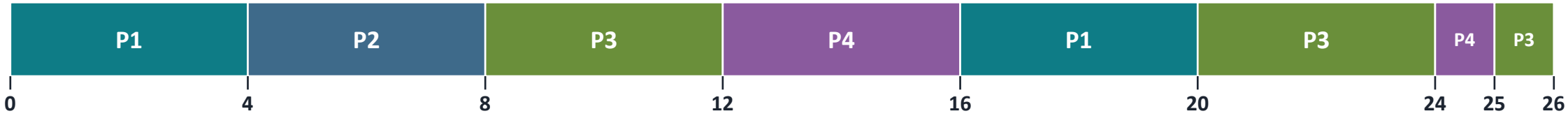
Zaman	Çalışan	Kalan	Aralık sonunda hazır kuyruğu (baş → son)
0–4	P1	4	P2 → P3 → P4 → P1
4–8	P2	0	P3 → P4 → P1
8–12	P3	5	P4 → P1 → P3
12–16	P4	1	P1 → P3 → P4
16–20	P1	0	P3 → P4
20–24	P3	1	P4 → P3
24–25	P4	0	P3
25–26	P3	0	boş

Bağlam değiştirme

7

8 parça → 7 geçiş (q = 2'de 12 idi). t = 4'te P2, P3, P4 zaten kuyruktadır; P1 sona eklenir.

RR (q = 4): Süre Hesapları



Süreç	Varış (AT)	Patlama (BT)	İlk başlama	Bitiş (CT)	TAT = CT − AT	WT = TAT − BT	RT = İlk − AT
P1	0	8	0	20	20	12	0
P2	1	4	4	8	7	3	3
P3	2	9	8	26	24	15	6
P4	3	5	12	25	22	17	9
Ortalama					**18,25**	**11,75**	**4,5**

Ort. TAT = (20 + 7 + 24 + 22) / 4 = 18,25

Ort. WT = (12 + 3 + 15 + 17) / 4 = 11,75

Ort. RT = (0 + 3 + 6 + 9) / 4 = 4,5

q = 2 ile karşılaştırma: ort. WT 12,75 → 11,75; ort. RT 2 → 4,5.

Zaman Dilimi Karşılaştırması: q = 2 ve q = 4

Ölçüt	RR q = 2	RR q = 4
Ortalama bekleme (WT)	12,75	11,75
Ortalama tamamlanma (TAT)	19,25	18,25
Ortalama yanıt (RT)	2	4,5
Bağlam değiştirme sayısı	12	7

Küçük q

- Daha iyi yanıt süresi potansiyeli
- Daha fazla bağlam değiştirme ve ek yük

Büyük q

- Daha az geçiş, daha az ek yük
- Davranış FCFS'ye yaklaşır

Bu iş yükünde q = 4, WT ve TAT'ta biraz daha iyi; q = 2 ise yanıt süresinde daha iyidir. Hangisinin “daha iyi” olduğu hangi ölçütü önemsediğimize bağlıdır.

Bağlam değiştirme süresi 0 kabul edilmiştir; gerçekte her geçiş ek süre ekler.

Zaman Dilimi Boyutunun Etkisi

Kitap örneği: 10 birimlik tek bir süreç



0 bağlam değiştirme

1 bağlam değiştirme

9 bağlam değiştirme

Genel ilke

q, bağlam değiştirme süresine göre büyük olmalıdır; ama çok büyük de olmamalıdır.

q çok büyük

Süreçlerin çoğu tek dilimde biter → RR, FCFS gibi davranır.

q çok küçük

Çok sayıda bağlam değiştirme; geçiş süresi dilime göre anlamlı büyüklükteyse CPU zamanının önemli kısmı ek yüke gider.

Kaynak: Silberschatz vd., Bölüm 6.3.4 (Şekil 6.4) · Kitaptaki önerilen eğilim: CPU patlamalarının yaklaşık %80'i q'dan kısa olmalıdır

Aynı İş Yükü, Yedi Çözüm: Karşılaştırma

Algoritma	Ort. WT	Ort. TAT	Ort. RT	Kesintili?	Açlık riski	Güçlü yanı	Zayıf yanı
FCFS	8,75	15,25	8,75	Hayır	Yok	Basit, adil sıra	Konvoy etkisi
SJF	7,75	14,25	7,75	Hayır	Var	En küçük ort. WT (kesintisizler arasında)	Patlama süresi bilinmez
SRTF	6,5	13	4,25	Evet	Var	En küçük ort. WT	Kalan süre tahmini, daha çok geçiş
Öncelik (kesintisiz)	7,75	14,25	7,75	Hayır	Var	Önemli işler öne	Açlık
Öncelik (kesintili)	7	13,5	6	Evet	Var	Yüksek öncelik hemen çalışır	Açlık
RR q = 2	12,75	19,25	2	Evet	Yok	En iyi yanıt süresi	Yüksek TAT, çok geçiş
RR q = 4	11,75	18,25	4,5	Evet	Yok	Yanıt ve ek yük dengesi	Yüksek TAT

Bu iş yükünde: en düşük ort. WT ve TAT → SRTF · en düşük ort. RT → RR q = 2. Tüm değerler önceki çözüm slaytlarıyla aynıdır.

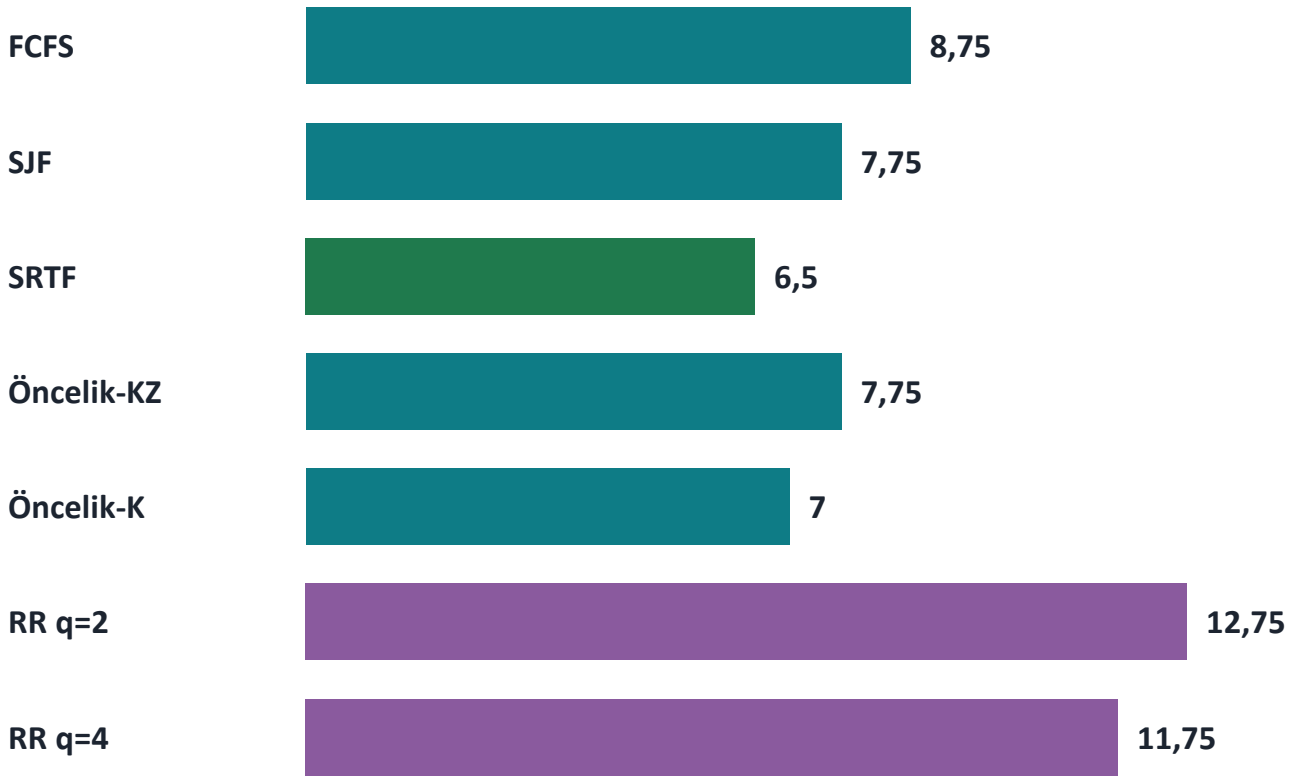
Deterministik Modelleme ile Değerlendirme

- Önceden belirlenmiş bir iş yükü alınır
- Her algoritma bu iş yüküyle çalıştırılır
- Ölçütler hesaplanıp karşılaştırılır
- Bu hafta yaptığımız tam olarak budur

Sınırlaması

Sonuçlar yalnızca o iş yükü için geçerlidir. Genel sonuç için kuyruk modelleri, benzetim ve gerçekleştirme kullanılır (bu derste ayrıntıya girilmez).

Ortalama bekleme süresi (WT)



KZ: kesintisiz · K: kesintili

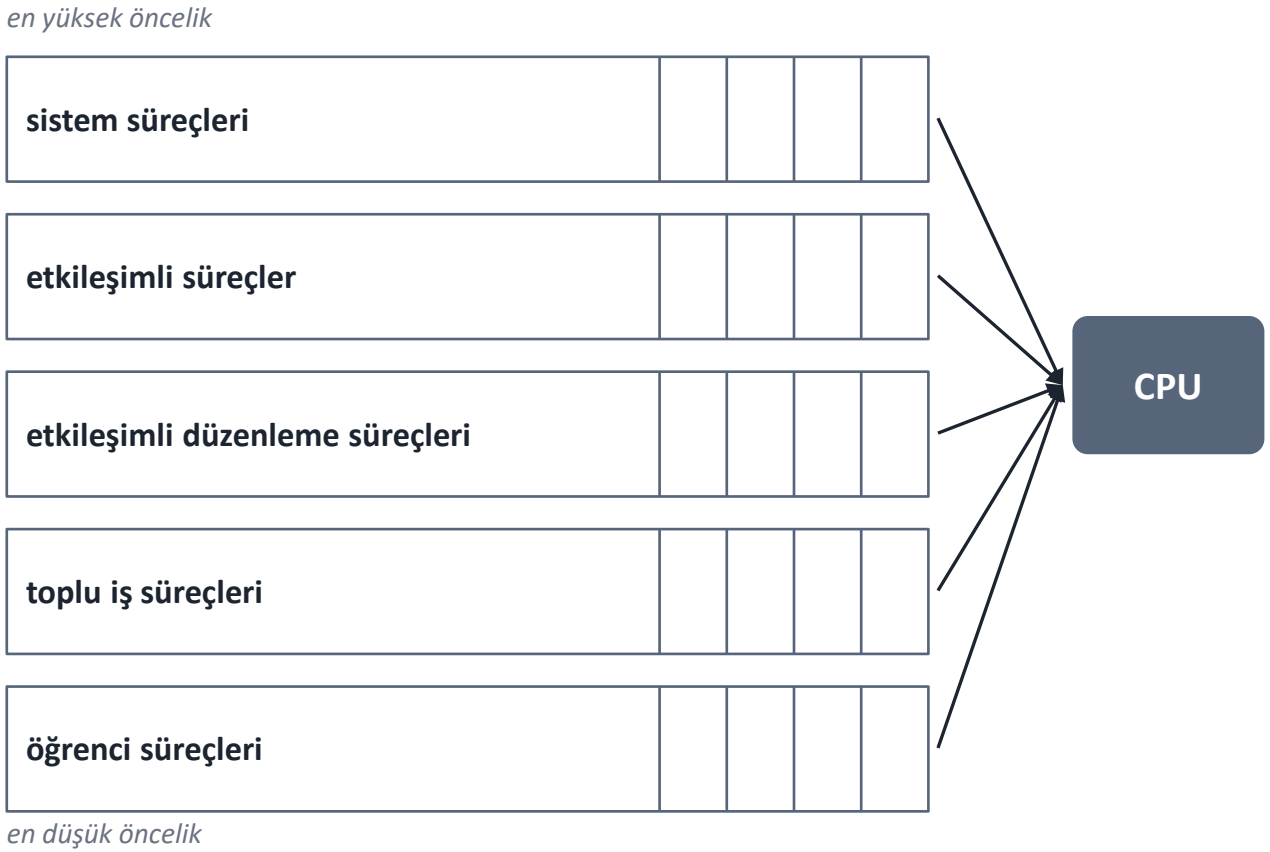
Kaynak: Silberschatz vd., Bölüm 6.8.1

Hangi Durumda Hangi Algoritma?

Toplu iş, süreler biliniyor	**SJF / SRTF**	Ortalama bekleme ve tamamlanma en düşük
Etkileşimli, çok kullanıcılı	**Round Robin**	Herkes düzenli aralıklarla CPU alır; yanıt süresi kısa
Süreçlerin önem düzeyi farklı	**Öncelik + yaşlandırma**	Önemli işler öne; açlık önlenir
Basitlik ve öngörülebilir sıra	**FCFS**	Gerçekleştirmesi kolay; ama konvoy etkisine açık
Karışık iş yükü	**Çok düzeyli (geri beslemeli) kuyruk**	Farklı süreç türleri farklı kuyruklarda → sonraki bölüm

Tek bir “en iyi” algoritma yoktur: seçim, sistemin hedeflerine (verim, yanıt süresi, adalet, öngörülebilirlik) bağlıdır.

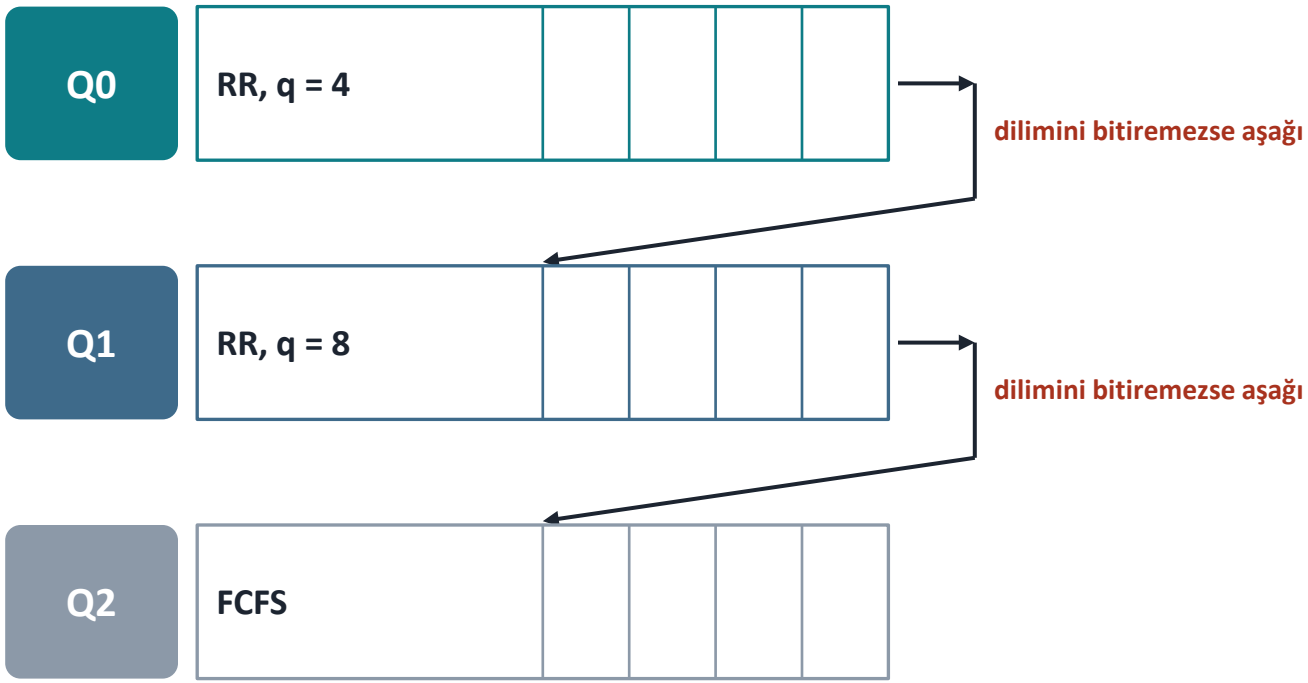
Çok Düzeyli Kuyruk (Multilevel Queue)



- Hazır kuyruğu birden çok ayrı kuyruğa bölünür (ör. ön plan / etkileşimli ve arka plan / toplu)
- Süreç türüne göre kalıcı olarak bir kuyruğa atanır
- Her kuyruğun kendi algoritması olabilir (ör. ön plan RR, arka plan FCFS)
- Kuyruklar arası: sabit öncelik (üst kuyruk boşalmadan alt çalışmaz) ya da zaman bölüştürme (ör. %80 ön plan, %20 arka plan)

Silberschatz vd., Bölüm 6.3.5 ve Şekil 6.6 temel alınmıştır

Çok Düzeyli Geri Beslemeli Kuyruk (MLFQ)



yukarı: en yüksek öncelik · aşağı: en düşük

- Süreçler davranışlarına göre kuyruklar arasında hareket edebilir
- Yeni süreç Q0'a girer; dilimini bitiremezse bir alt kuyruğa iner
- Alt kuyruk yalnızca üst kuyruklar boşken çalışır; üst kuyruğa gelen süreç alttakini keser
- Uzun süre alt kuyrukta bekleyen süreç yükseltilebilir (yaşlandırma / öncelik yükseltme)

q = 4, q = 8 ve FCFS değerleri öğretim amaçlı örnektir; evrensel bir sistem kuralı değildir. Silberschatz vd., Bölüm 6.3.6

MLFQ Neden Kullanılır? İki Sürecin Yolculuğu

Etkileşimli süreç E: CPU patlaması 3

Q0: 3 birim çalışır, dilim (4) dolmadan G/Ç'ye gider

→ *Q0'da kalır; her seferinde hızlı yanıt alır*

CPU ağırlıklı süreç U: CPU patlaması 30



4 + 8 + 18 = 30 birim; üst kuyruklar boşaldıkça ilerler

Tasarım parametreleri

- Kuyruk sayısı
- Her kuyruğun algoritması
- Bir süreç ne zaman yükseltilir / indirilir?
- Yeni süreç hangi kuyruğa girer?

Kazanım

Kısa, etkileşimli işler hızlı yanıt alır; uzun işler de sonunda ilerler.

Dikkat

Üst kuyruklar hiç boşalmazsa alt kuyruktakiler açlık yaşayabilir → yükseltme mekanizması gerekir.

İş Parçacığı Zamanlama: Çekişme Kapsamı

Hafta 4 hatırlatma: İS, süreçleri değil çekirdek iş parçacıklarını zamanlar; kullanıcı iş parçacıkları kütüphane tarafından yönetilir.

Süreç çekişme kapsamı (PCS)

- Aynı sürecin iş parçacıkları arasında yarış
- Kütüphane, kullanıcı iş parçacıklarını uygun LWP'lere yerleştirir
- Çoktan-bire ve çoktan-çoka modellerde

Sistem çekişme kapsamı (SCS)

- Sistemdeki tüm iş parçacıkları arasında yarış
- Çekirdek, hangi çekirdek iş parçacığının CPU'yu alacağına karar verir
- Birebir modelli sistemler (Windows, Linux) yalnızca SCS kullanır

```
pthread_attr_setscope(&attr, PTHREAD_SCOPE_SYSTEM);   /* SCS */
pthread_attr_setscope(&attr, PTHREAD_SCOPE_PROCESS);  /* PCS */
```

Kaynak: Silberschatz vd., Bölüm 6.4 · Bazı sistemler (ör. Linux) yalnızca PTHREAD_SCOPE_SYSTEM'e izin verir

Çok İşlemcili Zamanlama: Yeni Soru

Birden çok CPU / çekirdek varsa: hangi iş parçacığı hangi işlemcide çalışacak?

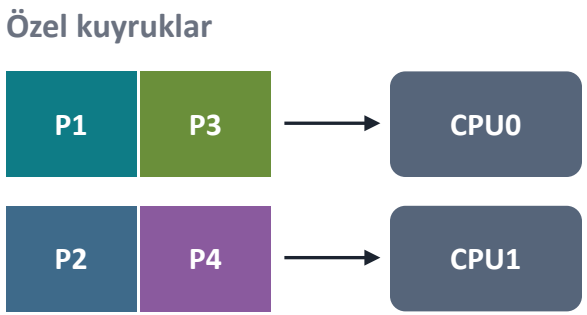
Asimetrik çok işlemcilik

Tüm zamanlama kararlarını ve sistem işlerini tek bir ana işlemci yapar; diğerleri yalnızca kullanıcı kodunu yürütür. Basittir, ama ana işlemci darboğaz olabilir.

Simetrik çok işlemcilik (SMP)

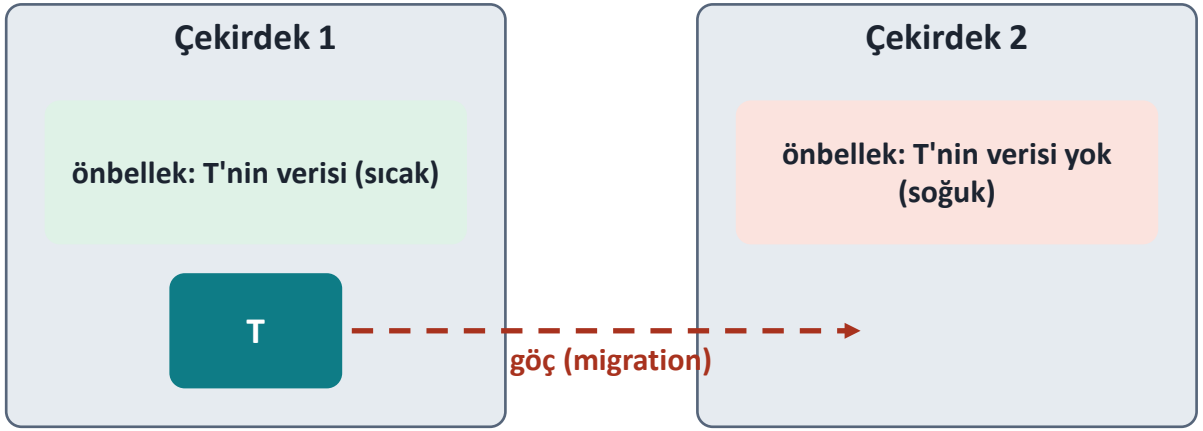
Her işlemci kendi zamanlamasını yapar. Hazır süreçler ortak bir kuyrukta ya da her işlemcinin kendi özel kuyruğunda olabilir. Modern sistemlerin çoğu SMP'dir.





Ortak kuyrukta iki işlemcinin aynı süreci seçmemesi sağlanmalıdır (eşgüdüm: Hafta 6). Kaynak: Silberschatz vd., Bölüm 6.5.1

İşlemci Yakınlığı (Processor Affinity)



- İş parçacığı çalıştığı işlemcinin önbelleğini kendi verisiyle doldurur
- Başka işlemciye taşınırsa eski önbellek geçersiz kılınmalı, yenisi baştan doldurulmalıdır → maliyet
- Bu yüzden İS süreci aynı işlemcide tutmaya çalışır

Yumuşak yakınlık (soft affinity)

İS aynı işlemcide tutmaya çalışır ama garanti etmez; süreç göç edebilir.

Katı yakınlık (hard affinity)

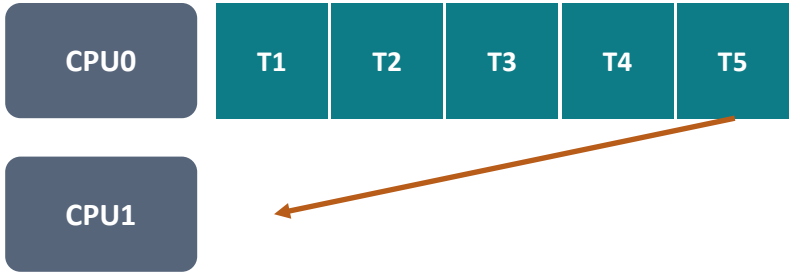
Süreç, çalışabileceği işlemcileri sistem çağrılarıyla belirleyebilir (ör. Linux: sched_setaffinity()).

Kaynak: Silberschatz vd., Bölüm 6.5.2

Yük Dengeleme (Load Balancing)

Amaç: Özel kuyruklu SMP sistemlerinde işin işlemciler arasında dengeli dağılması — bir işlemci boşta, diğeri aşırı yüklü kalmamalı.

İtme göçü (push migration)



Bir görev yükleri düzenli olarak denetler ve dengesizlikte yükü boş işlemciye iter

Çekme göçü (pull migration)



Boşta kalan işlemci, meşgul işlemciden bekleyen görevi kendisi çeker

Ödünleşim: Yük dengeleme süreçleri taşır, yakınlık ise yerinde tutmak ister. Taşımanın önbellek maliyeti, dengelemenin kazancıyla karşılaştırılmalıdır.

İki yaklaşım birlikte de kullanılabilir. Kaynak: Silberschatz vd., Bölüm 6.5.3

Gerçek Zamanlı Zamanlama: Yumuşak ve Katı

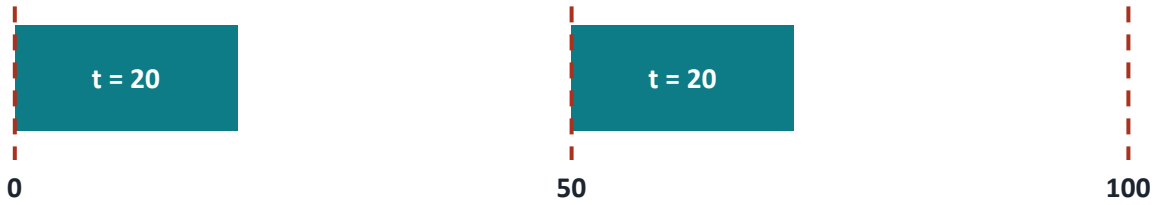
Yumuşak gerçek zamanlı (soft)

Kritik süreçlere öncelik verilir, ama ne zaman çalışacakları garanti edilmez.

Katı gerçek zamanlı (hard)

Görev son tarihine (deadline) kadar hizmet almalıdır; süresi geçmiş hizmet hizmet almamakla eşdeğerdir.

Periyodik görev modeli



periyot p = 50, işlem süresi t = 20, son tarih = periyot sonu

- Görev her p zaman biriminde bir CPU ister
- $0 \leq t \leq d \leq p$
- CPU kullanım oranı = t / p (örnek: 20/50 = 0,4)

Kaynak: Silberschatz vd., Bölüm 6.6 · Bu bölüm yalnızca kısa bir giriştir

Hız Monoton (RM) ve En Erken Son Tarih Önce (EDF)

Hız Monoton (Rate-Monotonic)

- Statik öncelik: periyodu kısa olan görev yüksek öncelik alır
- Kesintilidir
- Statik öncelikli algoritmalar arasında optimaldir; ama CPU kullanımı için bir üst sınırı vardır

En Erken Son Tarih Önce (EDF)

- Dinamik öncelik: son tarihi en yakın olan görev çalışır
- Görevlerin periyodik olması gerekmez
- Kuramsal olarak CPU'yu %100 kullanabilir (bağlam değiştirme ve kesme maliyetleri yok sayıldığında)

Kitap örneği: P1 (p = 50, t = 20) ve P2 (p = 100, t = 35) → toplam kullanım 20/50 + 35/100 = 0,75. RM'de P1 yüksek önceliklidir ve her iki görev de son tarihlerini karşılar.

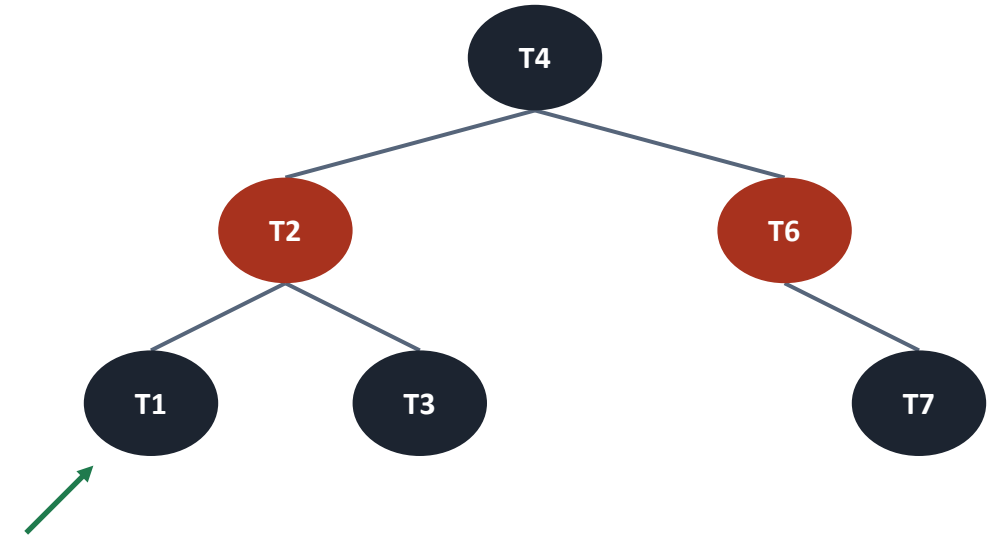
Kanıtlar ve ayrıntılı hesaplar bu dersin kapsamı dışındadır. Kaynak: Silberschatz vd., Bölüm 6.6.3–6.6.4

Kitap Temelli Sistem Örneği: Linux CFS

KİTAP TEMELLİ SİSTEM ÖRNEĞİ

- CFS (Completely Fair Scheduler): Linux 2.6.23 ile varsayılan zamanlama algoritması
- Sabit zaman dilimi yerine her göreve CPU zamanının bir payı verilir; pay, nice değerine (−20 … +19, küçük = yüksek öncelik) göre belirlenir
- Her görevin sanal çalışma süresi (vruntime) tutulur; düşük öncelikli görevlerin vruntime değeri daha hızlı artar
- Sıradaki görev: vruntime değeri en küçük olan

Çalıştırılabilir görevler: vruntime anahtarlı kırmızı-siyah ağaç



Silberschatz vd., Bölüm 6.7.1 (9. baskı, 2013) · Güncel Linux sürümlerindeki değişiklikler bu sunumda ele alınmamıştır.

Kitap Temelli Sistem Örneği: Windows Zamanlama

KİTAP TEMELLİ SİSTEM ÖRNEĞİ

- Öncelik tabanlı, kesintili iş parçacığı zamanlama: en yüksek öncelikli hazır iş parçacığı çalışır
- Zamanlamayı yapan çekirdek bölümüne dağıtıcı (dispatcher) denir
- Çalışan iş parçacığı; daha yüksek öncelikli biri gelene, bitene, dilimi dolana ya da engelleyen bir çağrı yapana kadar çalışır
- Hazır iş parçacığı yoksa boşta (idle) iş parçacığı çalıştırılır

32 düzeyli öncelik şeması



Silberschatz vd., Bölüm 6.7.2 (kitaptaki Windows 7 anlatımı) · Güncel Windows ayrıntıları eklenmemiştir.

Bu Hafta Neler Öğrendik?

1. CPU Zamanlayıcısı kimi, dağıtıcı nasıl sorusunu yanıtlar; Donanım Zamanlayıcısı kesme üretir
2. TAT = CT − AT · WT = TAT − BT · RT = ilk başlama − AT
3. FCFS basit ama konvoy etkisine açık; SJF ort. WT'de optimal ama patlama süresi tahmin edilmeli
4. SRTF ve kesintili öncelik her varışta yeniden karar verir; öncelikte açlığa karşı yaşlandırma
5. RR'de q küçüldükçe yanıt iyileşir, geçiş ek yükü artar; çok büyük q → FCFS
6. MLFQ, çok işlemcide yakınlık ve yük dengeleme; Linux CFS, Windows öncelik şeması

Gelecek hafta: Kesintili zamanlamada bir süreç her an kesilebilir. Süreçler veri paylaşıyorsa ne olur? → Süreç senkronizasyonu

Yaygın Yanlış Anlamalar

YANLIŞ	DOĞRU
“Bekleme süresi = yanıt süresi”	WT hazır kuyruğundaki toplam bekleme; RT yalnızca ilk CPU'ya kadar geçen süre
“SJF her zaman uygulanabilir”	Sonraki patlama süresi bilinmez; ancak tahmin edilebilir
“SRTF kesintisizdir”	SRTF, SJF'nin kesintili sürümüdür
“Küçük zaman dilimi her zaman daha iyidir”	Yanıt iyileşir ama bağlam değiştirme ek yükü artar
“Büyük öncelik sayısı her zaman yüksek önceliktir”	Sisteme bağlıdır: bu derste küçük sayı yüksek; Windows'ta büyük sayı yüksek
“RR'de bitmeyen süreç kuyruğun başına döner”	Kuyruğun SONUNA eklenir
“CPU Zamanlayıcısı = dağıtıcı”	Zamanlayıcı kimi seçer; dağıtıcı CPU'yu ona verir
“CPU Zamanlayıcısı = Donanım Zamanlayıcısı”	Biri yazılım kararı, diğeri kesme üreten donanım

DEĞERLENDİRME

Kontrol Soruları

KAVRAMSAL

S1 **CPU Zamanlayıcısı, dağıtıcı ve Donanım Zamanlayıcısı bir RR sisteminde bir zaman dilimi dolduğunda sırasıyla ne yapar?**

İlgili bölüm: 1 · Temel kavramlar

S2 **Kesintisiz bir algoritmada yanıt süresi neden bekleme süresine eşittir? Kesintili algoritmada neden farklı olabilir?**

İlgili bölüm: 2 · Zamanlama ölçütleri

Kontrol Soruları

FCFS · SJF HESABI

Süreç	Varış	CPU patlaması
A	0	6
B	2	2
C	3	8
D	5	3

S3 **Yandaki iş yükü için FCFS ve kesintisiz SJF Gantt şemalarını çiziniz; her algoritmada ortalama bekleme süresini hesaplayınız.**

Varış zamanlarına dikkat: henüz gelmemiş süreç seçilemez.

S4 **İki algoritmada D sürecinin tamamlanma süresi (TAT) ne kadardır? Neden farklıdır?**

TAT = CT − AT

Kontrol Soruları

SRTF · ÖNCELİK

Süreç	Varış	CPU patlaması
A	0	6
B	2	2
C	3	8
D	5	3

S5

Aynı iş yükünü SRTF ile çözünüz. Hangi anlarda kesme olur? Her sürecin yanıt süresini bulunuz.

Her varışta kalan süreleri karşılaştırın.

S6

Öncelikler A = 3, B = 1, C = 4, D = 2 (küçük sayı yüksek) olsun. Kesintili öncelik zamanlamada ortalama bekleme süresi nedir?

Eşitlikte FCFS.

Kontrol Soruları

ROUND ROBIN ANALİZİ

Süreç	Varış	CPU patlaması
A	0	6
B	2	2
C	3	8
D	5	3

S7 **Aynı iş yükünü RR (q = 3) ile çözünüz. Her zaman dilimi sonundaki hazır kuyruğunu yazınız; ortalama WT ve RT'yi hesaplayınız.**

Aynı anda gelen yeni süreç, kesilen süreçten önce kuyruğa eklenir.

S8 **q = 3 yerine q = 10 seçilseydi Gantt şeması hangi algoritmanınkiyle aynı olurdu? Neden?**

İpucu: En uzun patlama kaç birim?

Kontrol Soruları

ALGORİTMA SEÇİMİ

S9 **Bir üniversite sunucusunda hem etkileşimli kullanıcı oturumları hem de uzun toplu işler çalışıyor. Hangi zamanlama yaklaşımını önerirsiniz? Gerekçelendiriniz.**

Tek kuyruk mu, çok düzeyli kuyruk mu?

S10 **α = 0,5, önceki tahmin τ = 12 ms ve son CPU patlaması t = 8 ms ise sonraki patlamanın tahmini nedir? α = 1 olsaydı ne olurdu?**

$\tau(n+1) = \alpha \cdot t(n) + (1 - \alpha) \cdot \tau(n)$

Kaynaklar

ANA KAYNAK

Silberschatz, A., Galvin, P. B., & Gagne, G. (2013). *Operating System Concepts* (9th ed.). John Wiley & Sons.

Bu hafta: Bölüm 6 — 6.1–6.5; 6.6 (kısa); 6.7.1 Linux, 6.7.2 Windows; 6.8.1 deterministik modelleme

DESTEKLEYİCİ KAYNAK

Tanenbaum, A. S., & Bos, H. (2015). *Modern Operating Systems* (4th ed.). Pearson.

Bu hafta: Bölüm 2.4 — 2.4.1–2.4.6

Not: Güncel dış kaynak kullanılmamıştır; Linux ve Windows örnekleri kitap temellidir. Gantt şemaları ve tüm hesaplar ortak iş yükü üzerinden yeniden üretilmiştir.